



STUDENT RECORD MANGEMENT SYSTEM

Student :

**Reman alsaif
Walaa alenazi
Remas alharbi
Shatha ali**

Dr . Jawaher Alharbi

1. SYSTEM DESIGN AND ARCHITECTURE

In this project we built the Student Management System as a console-based Java application. Instead of putting all the code in one place we divided the system into several classes so that each part handles a specific responsibility. This helped us keep the project more organized and made the logic easier to understand

. The main idea of the design is that different classes work together without depending too much on each other. Below is a simple overview of how the system is structured:

- **Student (abstract class):** It contains the common fields used by all students, such as ID, name, department, year, and GPA. It also defines abstract methods that each student type must implement.
- **UnderGrad and GradStudent:** These two classes extend Student and provide their own versions of the abstract methods. The difference between them appears mainly in scholarship calculation and student type.
- **RecordManager:** This class handles almost all the operations in the system. It stores the list of students and provides functions for adding, updating, deleting, searching, sorting, and calculating averages. It also loads and saves the data to a CSV file.
- **AutoSaver and ReportBuilder:** These classes run in separate threads. AutoSaver saves the data automatically every few seconds, while ReportBuilder generates a report that includes sorted students and department averages.
- **Main:** The program starts here. It loads data from the file, runs the background AutoSaver, displays the menu and connects the user's choices to the correct functions.

By using inheritance, abstraction, custom exceptions, and multithreading, the system becomes more organized and easier to improve in the future. This structure also helped us keep our code cleaner while working as a team.

2. CLASSES, METHODS, AND FEATURES IMPLEMENTED

The system includes several classes and each one has a specific purpose. Below is a simplified explanation of the main classes and the functions they provide:

1. Student, UnderGrad, and GradStudent The Student class stores the general information for all students. The subclasses UnderGrad and GradStudent extend it and implement the abstract methods in their own way, mainly for scholarship calculation and student type.

2. StudentException A custom exception used when an operation fails such as trying to update or delete a student that does not exist. This makes error handling cleaner and more organized.

3. RecordManager This class handles all record operations in the application. It stores students in a list and provides the main functions:

- addStudent
- deleteStudent
- updateStudentGpa
- findStudentById
- getAllStudents
- getStudentsSortedByGpaDesc
- getAverageGpaByDepartment
- saveToFile / loadFromFile

4. AutoSaver and ReportBuilder Two background threads: - AutoSaver automatically saves the file at intervals. - ReportBuilder creates reports with sorted students and GPA averages.

5. Main Class – Program Features The Main class displays the menu, reads choices, and calls the needed methods. The program offers:

- *Adding both undergraduate and graduate students
- *Updating GPAs
- *Deleting students
- *Listing all records
- *Generating reports
- *Saving and loading from file
- *Automatic background saving

03

3.USE OF OOP PRINCIPLES AND EXCEPTION HANDLING

THE STUDENT RECORD MANAGEMENT SYSTEM (SRMS) DEMONSTRATES EFFECTIVE APPLICATION OF SEVERAL OBJECT-ORIENTED PROGRAMMING (OOP) PRINCIPLES. THESE PRINCIPLES WERE USED TO ENSURE CLEAN CODE ORGANIZATION, REUSABILITY, SCALABILITY, AND EASE OF MAINTENANCE

1-INHERITANCE:

INHERITANCE WAS APPLIED THROUGH THE CLASS HIERARCHY:

- **STUDENT** (ABSTRACT PARENT CLASS)
- **UNDERGRAD** AND **GRADSTUDENT** (CHILD CLASSES) BOTH SUBCLASSES INHERIT COMMON ATTRIBUTES SUCH AS ID, NAME, DEPARTMENT, GPA, AND OVERRIDE REQUIRED METHODS SUCH AS `CALCULATESCHOLARSHIPAMOUNT()`.

2-POLYMORPHISM :

POLYMORPHISM IS SHOWN THROUGH THE OVERRIDDEN METHODS IN THE SUBCLASSES.

FOR EXAMPLE, BOTH UNDERGRAD AND GRADSTUDENT IMPLEMENT DIFFERENT VERSIONS OF:

- **`CALCULATESCHOLARSHIPAMOUNT()`**
- **`GETTYPE()`**

THIS ALLOWS THE SYSTEM TO TREAT ALL STUDENTS AS “STUDENT” OBJECTS WHILE EXECUTING BEHAVIOR BASED ON THEIR ACTUAL TYPE.

3- ABSTRACTION:

THE STUDENT CLASS IS DESIGNED AS AN ABSTRACT CLASS TO DEFINE SHARED STRUCTURE AND BEHAVIOR ACROSS ALL STUDENT TYPES. IT INCLUDES:

- ABSTRACT METHODS
- COMMON ATTRIBUTES
- SHARED CONSTRUCTORS

THIS ENSURES THAT ONLY VALID STUDENT TYPES (UG/GRADUATE) CAN BE INSTANTIATED

4- ENCAPSULATION:

ALL ATTRIBUTES INSIDE THE CLASSES ARE DECLARED **PRIVATE** TO PROTECT THE DATA. ACCESS TO ATTRIBUTES IS CONTROLLED THROUGH:

- **GETTERS AND SETTERS**
- **VALIDATION INSIDE SETTER METHODS WHEN NEEDED**

THIS ENHANCES DATA INTEGRITY AND MINIMIZES UNINTENDED MODIFICATION.

5-COMPOSITION:

THE **RECORDMANAGER** CLASS MAINTAINS A **LIST**, REPRESENTING “HAS-A” RELATIONSHIP. RECORDMANAGER IS RESPONSIBLE FOR:

- ADDING AND REMOVING STUDENTS
- UPDATING GPA
- GENERATING REPORTS

THIS COMPOSITION STRUCTURE KEEPS STUDENT DATA ORGANIZED AND CENTRALIZED.

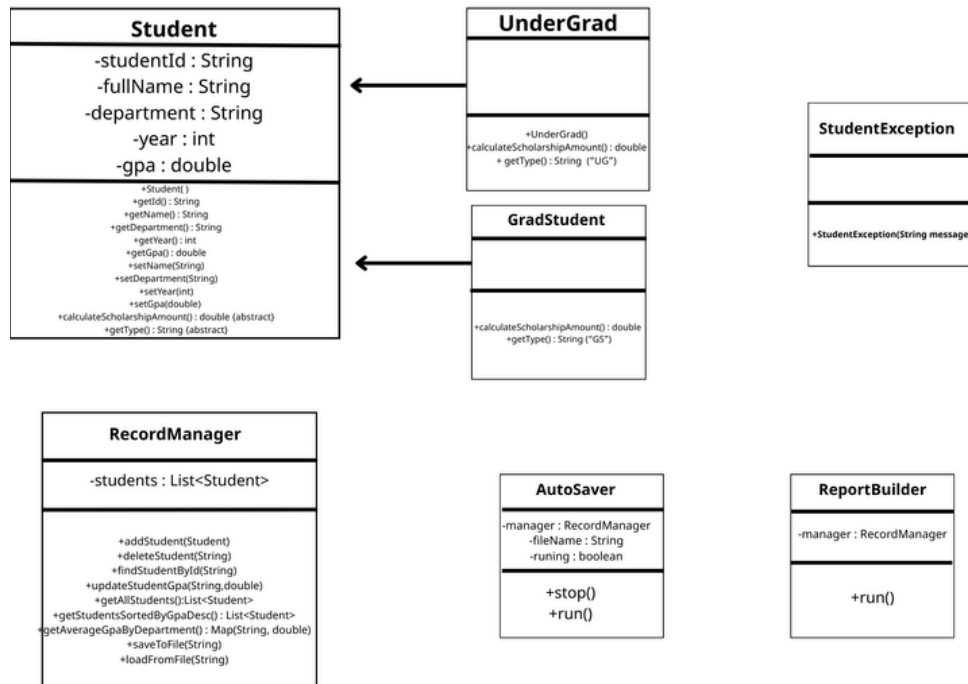
6-EXCEPTION HANDLING:

A DEDICATED CUSTOM EXCEPTION (**STUDENTEXCEPTION**) WAS IMPLEMENTED TO HANDLE INVALID CASES, SUCH AS:

- TRYING TO UPDATE OR DELETE A STUDENT WHO DOES NOT EXIST
- INVALID INPUT FORMATS
- PROCESSING ERRORS DURING FILE OPERATIONS

EXCEPTION HANDLING IMPROVES THE ROBUSTNESS OF THE SYSTEM BY PREVENTING PROGRAM CRASHES AND PROVIDING CLEAR ERROR MESSAGES.

OVERALL, THE PROJECT USES OOP TO IMPROVE MODULARITY, REDUCE REDUNDANCY, AND ORGANIZE THE PROGRAM INTO CLEAR, MAINTAINABLE COMPONENTS.



The UML Class Diagram provides a structural view of the system's design, illustrating the relationships between classes and how they interact. The diagram includes:

1-Student (Abstract Class):

- **Attributes:** id, name, department, GPA
- **Methods:** getters/setters, abstract methods (calculateScholarshipAmount, getType)

2-UnderGrad (Subclass of Student):

- Inherits from Student
- Overrides:
- calculateScholarshipAmount()
- getType()

3-GradStudent (Subclass of Student):

- Inherits from Student
- Overrides scholarship and type methods
- Different GPA rules compared to UG

4-RecordManager:

- Uses composition (List)
- **Methods:** addStudent , deleteStudent , updateGPA , getAllStudents , sortByGPA , calculateDepartmentAverage

5-AutoSaver (Thread):

- Extends Thread
- Runs in background to save data automatically

6-ReportBuilder (Thread):

- Generates GPA reports without interrupting main program execution

7-StudentException:

- Custom exception class
- Handles invalid operations

Relationship Summary:

- Inheritance: Student → UnderGrad, GradStudent
- Composition: RecordManager → List
- Association: Threads interact with RecordManager
- Dependency: File handling depends on RecordManager operations

5. CHALLENGES ENCOUNTERED AND SOLUTIONS

Challenge 1: Handling File Consistency During Concurrent Tasks When implementing multithreading (e.g., autosave + report generation), the system initially faced file-access conflicts resulting in partial writes or locked files.

Solution: We implemented thread synchronization using synchronized blocks to ensure controlled access to shared resources. This prevented race conditions and maintained data integrity during simultaneous operations.

Challenge 2: Designing a Flexible and Scalable Class Structure At first, the Student class and related components became tightly coupled, making it difficult to extend the system.

Solution: We redesigned the architecture using proper OOP principles, especially inheritance, composition, and encapsulation. This allowed clear separation of responsibilities and made future modifications easier.

Challenge 3: Validating User Input and Handling Exceptions The system frequently crashed when receiving unexpected or malformed input (e.g., missing fields, invalid GPA format).

Solution: We added comprehensive input validation and applied Java exception handling mechanisms (try-catch, custom exceptions). This improved system stability and error reporting.

Challenge 4: Managing GitHub Collaboration During development, merge conflicts occurred frequently due to multiple branches modifying the same files.

Solution: We established a workflow using feature branches, required pull-request reviews, and consistent commit messages to maintain clean version history and avoid conflicts.

Challenge 5: Generating Detailed Reports Efficiently Report generation (by GPA, department, and year) was initially slow when handling large datasets.

Solution: We optimized sorting using Java's built-in Comparator and stored repeated calculations in temporary structures, reducing the execution time significantly.

6. TEAM COLLABORATION APPROACH USING GITHUB

Our team used GitHub to organize and manage the project efficiently. Each member was responsible for specific tasks such as writing the main code, preparing the UML diagram, and documenting the project in the README file. We used GitHub repositories to store all project files and keep them updated. Branches were used to work on different parts of the project separately and then merge them into the main branch. Commits helped us track changes made by each member, and GitHub was also used to share files, review work, and avoid conflicts. This collaboration approach helped us divide the work clearly and complete the project in an organized way.

7.SUGGESTIONS FOR FUTURE IMPROVEMENTS

- 1-Add a simple GUI (JavaFX/Swing) To make the system easier to use instead of relying on console-based interaction.
- 2-Move from text files to a database Improves speed, scalability, and data reliability as the number of students grows.
- 3-Include performance analytics Such as GPA distribution and department statistics to support academic decisions.
- 4-Add user roles and authentication To protect student records and limit access to authorized users only.
- 5-Enable automatic backups To prevent data loss and ensure safe recovery in case of system failure.

8.LEGAL AND ETHICAL CONSIDERATIONS IN OBJECT ORIENTED DESIGN

The OOP design of the SRMS supports legal and ethical data handling by using encapsulation to protect student information, exception handling to prevent invalid operations, and structured class responsibilities to ensure controlled access.

Inheritance and polymorphism maintain consistency and accuracy across all student records, while the system's file handling and multithreading preserve data integrity.

Together, these features ensure confidentiality, reliability, and secure storage of student data.

9.REFERENCES

- 1- THE DETAILED EXPLANATIONS PROVIDED BY THE GEEKSFORGEEKS WEBSITE GAVE US THE FOUR PRINCIPLES OF OBJECT-ORIENTED PROGRAMMING (ENCAPSULATION, INHERITANCE, POLYMORPHISM, AND ABSTRACTION) .
- 2- GITHUB BLOG: PUBLISHES ARTICLES ON BEST PRACTICES FOR DOCUMENTATION AND TEAM WORKFLOWS USING GITHUB FEATURES
- 3- DSPACE.ORG: EXAMPLES OF REPORTS FOR ACADEMIC PROJECTS OFTEN INCLUDE SECTIONS DEDICATED TO THE CHALLENGES FACED BY THEIR TEAM.
- 4- [HTTPS://TSA.EDU.AU/BLOG/LEGAL-AND-ETHICAL-CONSIDERATIONS-IN-COMMUNITY-SERVICES/](https://tsa.edu.au/blog/legal-and-ethical-considerations-in-community-services/)